AI Media Studio
User Manual

A fully local, AI-powered pipeline that turns one topic into a finished, narrated, subtitled short-form video. No cloud services and no paid APIs.

Version	0.1.0
Requires	Python 3.12 or newer
Runs on	Windows, verified on Windows 11
Output	1080x1920 H.264 MP4 with AAC audio

Contents

1 What this does

You give AI Media Studio a topic. It researches and writes a script, breaks it into ordered scenes, narrates them, generates an image for each scene, writes subtitles, and renders the whole thing into a vertical MP4 ready for Reels, Shorts or TikTok. Everything runs on your own machine.

The pipeline

```
Topic
  -> Script and ordered scenes            (Ollama)
  -> Narration, measured per scene        (Kokoro)
  -> Scene durations reconciled to speech
  -> One image per scene                  (ComfyUI)
  -> Subtitles                            (SRT)
  -> Final MP4                            (FFmpeg)
```

Each stage writes its results to a project folder as it goes. Nothing is held only in memory, which is what makes an interrupted run resumable (section 6).

What you need to supply

- A topic, as a short phrase.
- Optionally a style, a voice, background music, and whether you want subtitles.

2 Before you start

Four local services do the actual work. Install and start these first.

Component	Purpose	Notes
`Ollama`	Writes the script and plans scenes	Must be running. Pull a model first, for example `ollama pull llama3.1:8b`.
`ComfyUI`	Generates one image per scene	Must be running, with an image model installed. See section 11.
`Kokoro`	Speaks the narration	Installed automatically as a Python dependency. Downloads its voice on first use.
`FFmpeg`	Renders the final video	Either on your PATH or given as a full path in the configuration.

Hardware

Image generation is the demanding part. A machine with an 8 GB NVIDIA GPU produces an SDXL image in roughly 20 to 35 seconds once the model is loaded in memory, and 90 seconds or more before that. Less video memory still works but is slower, because the model is reloaded for each image.

3 Installing

```
pip install -e .
copy config\settings.example.toml config\settings.toml
```

Then open `config\settings.toml` and replace the two placeholder values: the Ollama model name and the Kokoro voice. Nothing will run until you do, because the placeholders are not real model names.

> **Installing without `-e`?** A non-editable install has no project folder to read from, so tell it where your settings file is with `--config` or the `AI_MEDIA_CONFIG` environment variable, and use absolute paths inside that file.

Checking it worked

```
ai-media-studio generate --help
```

4 Configuration

All settings live in one TOML file. By default that is `config\settings.toml` inside the project folder.

Choosing a different file

Highest priority first:

- The `--config` option on the command line.
- The `AI_MEDIA_CONFIG` environment variable.
- The project's own `config\settings.toml`.

The settings that matter most

Setting	What it controls	Default
`ollama.model`	Which local language model writes the script	none, you must set it
`kokoro.voice`	Which narration voice is used	none, you must set it
`kokoro.speed`	Speaking rate	1.0
`kokoro.scene_tail_padding_seconds`	Silence after each scene so speech does not butt against a cut	0.2
`paths.ffmpeg_executable`	FFmpeg command or full path	ffmpeg
`paths.output_dir`	Where project folders are created	output
`comfyui.workflow_path`	The ComfyUI workflow JSON to run	config/comfyui_workflow.json
`comfyui.timeout_seconds`	How long to wait for one image	120
`comfyui.max_retries`	Retries after a recoverable image failure	2
`video.width, video.height`	Output resolution	1080 by 1920
`video.frames_per_second`	Output frame rate	30
`video.transition_duration_seconds`	How long a transition lasts	0.5
`music.volume`	Background music level	0.15

Setting	What it controls	Default
`music.ducking_ratio`	How hard music dips under narration	6.0
`logging.level`	Detail written to the log	INFO

> **Raise `comfyui.timeout_seconds` to about 300.** The very first image of a run also has to load the image model into memory, which alone can take longer than the 120 second default. Otherwise the first scene times out and is retried needlessly.

Overriding settings without editing the file

Any of these environment variables takes precedence over the file, which is convenient for one-off changes and for scripting.

```
AI_MEDIA_CONFIG                     AI_MEDIA_LOG_LEVEL
AI_MEDIA_OLLAMA_BASE_URL            AI_MEDIA_LOG_DIRECTORY
AI_MEDIA_OLLAMA_MODEL               AI_MEDIA_LOG_CONSOLE_ENABLED
AI_MEDIA_OLLAMA_TIMEOUT_SECONDS     AI_MEDIA_LOG_FILE_ENABLED
AI_MEDIA_COMFYUI_BASE_URL           AI_MEDIA_CACHE_ENABLED
AI_MEDIA_COMFYUI_WORKFLOW_PATH      AI_MEDIA_CACHE_DIRECTORY
AI_MEDIA_COMFYUI_TIMEOUT_SECONDS    AI_MEDIA_TEMP_MAX_AGE_HOURS
AI_MEDIA_KOKORO_VOICE               AI_MEDIA_GPU_DEVICE
AI_MEDIA_KOKORO_SPEED               AI_MEDIA_GPU_MEMORY_LIMIT_MB
AI_MEDIA_VIDEO_WIDTH                AI_MEDIA_VIDEO_HEIGHT
AI_MEDIA_VIDEO_FPS                  AI_MEDIA_VIDEO_RENDER_TIMEOUT_SECONDS
AI_MEDIA_VIDEO_TRANSITION_DURATION_SECONDS
```

5 Generating a video

```
ai-media-studio generate --topic "Why Japan Never Sleeps"
```

With every option in use. The trailing backtick continues a command across lines in PowerShell; in Command Prompt use a caret instead.

```
ai-media-studio generate `
  --topic "Why Japan Never Sleeps" `
  --style "cinematic documentary" `
  --voice "af_heart" `
  --output "D:\Media Projects" `
  --music `
  --subtitle
```

Options

Option	Meaning
`--topic`	The subject of the video. Required unless you are resuming.
`--resume`	Continue an existing project folder instead of starting fresh. Cannot be combined with --topic.
`--style`	Narrative and visual style, for example *cinematic documentary*. Passed to the script writer.

Option	Meaning
`--voice`	Override the narration voice for this run only.
`--output`	Where the project folder is created, overriding the configured output folder.
`--config`	Use a different settings file.
`--music`	Pick a random track from your music folder and mix it under the narration.
`--subtitle`	Write subtitles and burn them into the video.

What you will see

```
[#######-------------] 1/3 Generating storyboard, narration, and images
[#############-------] 2/3 Generating subtitles
[####################] 3/3 Rendering final MP4

Generation complete
Generation time: 505.94 seconds
Provider versions: ollama=unknown
Final output: C:\AI-Media-Studio\output\20260808T164700953894Z_why-japan-never-sleeps\v
ideo\final.mp4
```

Roughly how long it takes

Measured on a laptop with an 8 GB NVIDIA GPU, for a six scene video.

Stage	Typical time	Notes
`Script`	15 to 60 seconds	Depends on the language model and topic.
`Narration`	80 to 115 seconds	Slower the first time, while the voice loads.
`Images`	20 to 95 seconds each	The first two are slowest, then it speeds up.
`Subtitles`	under a second	Written directly from the scene text.
`Rendering`	5 to 15 seconds	Longer with music and burned-in subtitles.

> A whole run is typically 6 to 10 minutes, and almost all of it is image generation. If a run fails near the end, resume it rather than starting over.

6 Resuming a run

Every stage saves its work as it finishes, so a run that stops partway through can carry on. This matters most when the image service becomes unavailable mid-run, which otherwise throws away a finished script and narration.

```
ai-media-studio generate --resume "output\20260808T164700953894Z_why-japan-never-sleeps
" --subtitle
```

When resuming:

- The script is read back from the project folder and never regenerated.
- Existing narration is reused as it is.
- Scene images already on disk are kept, and only missing ones are generated.
- The video is always rendered again, which is quick.

Resuming a project that is already complete regenerates nothing and simply re-renders, so it is a cheap way to try different music or subtitle settings without paying for image generation again.

7 Inside a project folder

Each run creates one timestamped folder under your output folder, named after the time and the topic.

```
20260808T164700953894Z_why-japan-never-sleeps\
  manifest.json        what ran, which providers, how long, and whether it succeeded
  storyboard.json      the script and every scene, with their final durations
  narration.wav        the spoken narration for the whole video
  subtitles.srt        subtitle cues, when --subtitle was used
  images\
    scene_001.png      one image per scene
    scene_002.png
  video\
    final.mp4          the finished video
  logs\
```

manifest.json is the place to look when something went wrong. It records which stage failed and why, and is written even for a failed run.

storyboard.json holds the narration text and the scene timings actually used. If you want to know why a video is the length it is, read this.

8 Scenes, motion and transitions

The script writer assigns each scene a camera motion and a transition into the next one. Only the values below are accepted; anything else the model invents is converted to the closest match, so an unusual word never spoils a run.

Camera motion

Value	Effect
`none`	The image is held still.
`zoom_in`	Slow push in towards the centre.
`zoom_out`	Slow pull back from the centre.
`pan, pan_right`	Drifts across the image to the right.
`pan_left`	Drifts across the image to the left.

Transitions

Value	Effect
`cut, none`	An instant change with no blend.
`fade, crossfade`	One scene blends into the next.
`dissolve`	A softer, more textured blend.
`wipeleft, wiperight`	The next scene slides across horizontally.
`wipeup, wipedown`	The next scene slides across vertically.

Transition length comes from `video.transition_duration_seconds`. A transition is automatically shortened if either neighbouring scene is too brief to accommodate it.

9 How timing works

This is worth understanding, because it explains why your video is the length it is.

The language model estimates how long each scene should take to say. Those estimates are unreliable and are never used to time the video. Instead the narration for each scene is generated first and measured, and those measurements become the timeline.

The result is that:

- The video is exactly as long as the narration. There is no trailing silence, and narration is never cut off mid-sentence.
- Each image is on screen for exactly as long as its own narration.
- Each transition begins the moment its scene stops speaking.
- Subtitles line up with the speech, because they use the same measurements.

> A consequence worth knowing: **you cannot set the video length directly.** It is whatever the narration takes. To get a shorter video, ask for a shorter script or raise `kokoro.speed`.

10 Background music

Put audio files in the folder named by `music.directory`, which is `music\` by default. Accepted formats are mp3, wav, m4a, aac, flac and ogg.

With `--music`, one track is chosen at random, looped to fit, faded in and out, and automatically dipped underneath the narration so the voice stays clear. Adjust the level with `music.volume` and the strength of the dip with `music.ducking_ratio`.

If the folder is missing or empty you get a warning and the video is still produced, just without music.

11 Setting up the ComfyUI workflow

Image generation runs whatever ComfyUI workflow you point it at, so you can use any model and any settings ComfyUI supports. Export your workflow from ComfyUI in API format and set `comfyui.workflow_path` to it.

Your workflow must satisfy two rules:

- Exactly one positive prompt node, found by following the sampler's positive input. The scene's image prompt is written into it for each image.
- Exactly one Save Image node, which is where the finished image is collected.

If either is missing or ambiguous you get a clear configuration error naming the problem, rather than a failed render later on. Nothing else about the workflow matters, so resolution, sampler, steps and model are entirely yours to choose.

> Generate portrait images that match your output aspect ratio, for example 768 by 1344 for a 1080 by 1920 video. Images are scaled and cropped to fit, so a landscape image will lose a lot of its edges.

12 Troubleshooting

The log file, by default `logs\ai_media_studio.log`, records every stage with timings and the reason for any failure.

Message you see	What it means and what to do
`Configuration file not found`	There is no settings file where it looked. Copy the example file to `config\settings.toml`, or pass `--config`.
`Generation failed during storyboard: HTTP Error 404`	Ollama is running but does not have the model named in your settings. Check `ollama list` and pull the model, or correct `ollama.model`.
`Generation failed during storyboard: connection_failed`	Ollama is not running, or is on a different address than `ollama.base_url`.
`Generation failed during image: Unable to reach ComfyUI`	ComfyUI is not running or has stopped. Start it, then resume the run with `--resume` so the script and narration are not lost.
`ComfyUI generation timed out`	Usually the first image of a run, while the model loads. Raise `comfyui.timeout_seconds` to about 300.
`No positive prompt node could be discovered`	Your workflow has no prompt node reachable from a sampler's positive input, or has several. See section 11.
`Multiple Save Image output nodes were discovered`	Your workflow saves more than one image. Leave exactly one Save Image node.
`Generation failed during render: FFmpeg is unavailable`	FFmpeg is not on your PATH. Set `paths.ffmpeg_executable` to its full path.
`Music warning, or Subtitle warning`	Not a failure. The video is still produced without that extra. Usually an empty or missing music folder.

Image generation is unusually slow

If every image takes 90 seconds rather than settling to around 25, the image model is being reloaded for each one because memory is tight. Close other applications, or use a smaller image model. Speed usually improves a few scenes into a run.

ComfyUI stops on its own during long runs

On machines with limited memory the image service can exit without an error. Restart it and resume the run, which keeps everything already generated.

13 Running the tests

The test suite uses only the Python standard library and never contacts a local model, so it runs in well under a second and is safe to run at any time.

```
python -m unittest discover -s tests -t .
```

Use it to confirm an installation is sound before spending time on a full run.